

DataLab — Semana 8

DML completo, filtros, operadores, ordenamiento y funciones de agregación.

Estudia los conceptos, revisa los ejemplos y resuelve los ejercicios. Descarga tus respuestas para conservarlas y entregarlas.

1. Propósito y objetivos

La meta es consultar, filtrar, ordenar, resumir e interpretar información de DataLab y modificar datos de forma segura.

- Utilizar SELECT, INSERT, UPDATE y DELETE.
- Aplicar WHERE, operadores de comparación, BETWEEN, IN, LIKE, NULL, AND, OR y NOT.
- Ordenar con ORDER BY y resumir con funciones de agregación.
- Usar GROUP BY y HAVING para responder preguntas de negocio.
- Verificar los cambios realizados.

Alcance: cada consulta trabaja con una sola tabla. No se permiten JOIN, subconsultas ni UNION, INTERSECT o EXCEPT. Sí se permiten varias condiciones dentro de un WHERE.

2. Preparación del entorno

1. Verifica la base datalab y las ocho tablas del modelo.
2. Si hay una carga previa, ejecuta s08-reset-datos.sql.
3. Ejecuta s08-datos-semilla.sql.
4. Verifica que cada tabla tenga al menos 100 filas.

Tabla	Columnas relevantes
cientifico_datos	id_cientifico, nombre, correo_institucional
proyecto	id_proyecto, nombre_proyecto, descripcion
dataset	id_dataset, nombre, fuente, fecha_carga, tamaño_filas
experimento	id_experimento, id_proyecto, id_cientifico, fecha_ejecucion, configuracion
modelo	id_modelo, id_experimento, nombre, version, algoritmo
metrica	id_metrica, id_modelo, nombre_metrica, valor, fecha_calculo
participacion	id_cientifico, id_proyecto
uso_dataset	id_dataset, id_experimento

Importante: el esquema no contiene estado en experimento ni notas en dataset. Para practicar NULL se usa dataset.tamaño_filas. metrica.valor admite valores entre 0 y 1.

3. SELECT: consultar datos

SELECT recupera información y no modifica los datos.

```
SELECT id_dataset, nombre, fuente
FROM dbo.dataset;
```

En consultas de trabajo, especifica las columnas necesarias en lugar de usar SELECT *.

Comando	Operación CRUD	Función
SELECT	Read	Consultar
INSERT	Create	Insertar
UPDATE	Update	Modificar
DELETE	Delete	Eliminar

4. WHERE y operadores

WHERE filtra filas que cumplen una condición.

```
SELECT id_metrica, nombre_metrica, valor
FROM dbo.metrica
WHERE valor >= 0.80;
```

Operador	Significado
= / <>	Igual / diferente
< / >	Menor / mayor
<= / >=	Menor o igual / mayor o igual
BETWEEN a AND b	Rango inclusivo
IN (...)	Lista de valores
LIKE 'EEG%'	Patrón; % equivale a cero o más caracteres y _ a uno
IS NULL / IS NOT NULL	Valor ausente / informado
AND / OR / NOT	Combinar o negar condiciones

BETWEEN

```
SELECT id_metrica, valor
FROM dbo.metrica
WHERE valor BETWEEN 0.70 AND 0.90;
```

IN

```
SELECT nombre, fuente
FROM dbo.dataset
WHERE fuente IN ('interna', 'externa');
```

LIKE

```
SELECT nombre
FROM dbo.dataset
WHERE nombre LIKE 'EEG%';
```

NULL

```
SELECT nombre, tamaño_filas
FROM dbo.dataset
WHERE tamaño_filas IS NULL;
```

AND exige que se cumplan todas las condiciones; OR permite que se cumpla alguna; NOT niega una condición. Si mezclas AND y OR, usa paréntesis para expresar la intención.

```
SELECT nombre, fecha_carga
FROM dbo.dataset
WHERE fuente = 'externa'
AND fecha_carga >= '2026-01-01';
```

5. ORDER BY

ASC ordena ascendentemente y DESC descendentemente. Con varios criterios, el primero tiene prioridad.

```
SELECT nombre, fuente, fecha_carga
FROM dbo.dataset
ORDER BY fuente ASC, fecha_carga DESC;
```

6. Funciones de agregación

Función	Propósito
COUNT(*)	Cuenta filas
SUM(columna)	Suma valores
AVG(columna)	Calcula promedio
MIN(columna)	Obtiene mínimo
MAX(columna)	Obtiene máximo

```
SELECT AVG(valor) AS promedio,
       MIN(valor) AS minimo,
       MAX(valor) AS maximo
FROM dbo.metrica;
```

Las agregaciones distintas de COUNT(*) normalmente ignoran valores NULL; SUM(tamano_filas) no incluye tamaños desconocidos.

7. GROUP BY y HAVING

GROUP BY crea grupos con valores compartidos. HAVING filtra los grupos según una agregación.

```
SELECT fuente, COUNT(*) AS cantidad_datasets  
FROM dbo.dataset  
GROUP BY fuente;
```

```
SELECT nombre_metrica, AVG(valor) AS promedio  
FROM dbo.metrica  
GROUP BY nombre_metrica  
HAVING AVG(valor) > 0.80;
```

WHERE vs. HAVING: WHERE filtra filas antes de agrupar; HAVING filtra grupos después de calcular agregaciones. AVG(valor) > 0.80 debe ir en HAVING.

8. INSERT, UPDATE y DELETE con seguridad

INSERT

```
INSERT INTO dbo.proyecto (nombre_proyecto, descripcion)
VALUES (N'Proyecto de práctica S08',
        N'Registro temporal para practicar DML.');
```

SQL Server genera id_proyecto porque es IDENTITY. Verifica el registro con SELECT.

UPDATE

```
SELECT id_proyecto, nombre_proyecto, descripcion
FROM dbo.proyecto
WHERE nombre_proyecto = N'Proyecto de práctica S08';
```

```
UPDATE dbo.proyecto
SET descripcion = N'Descripcion actualizada durante la practica S08.'
WHERE nombre_proyecto = N'Proyecto de práctica S08';
```

DELETE

```
SELECT id_proyecto, nombre_proyecto
FROM dbo.proyecto
WHERE nombre_proyecto = N'Proyecto de práctica S08';
```

```
DELETE FROM dbo.proyecto
WHERE nombre_proyecto = N'Proyecto de práctica S08';
```

```
SELECT id_proyecto, nombre_proyecto
FROM dbo.proyecto
WHERE nombre_proyecto = N'Proyecto de práctica S08';
```

Regla de seguridad: ejecuta primero el SELECT equivalente, confirma las filas afectadas y vuelve a consultar después. Sin WHERE, UPDATE o DELETE puede afectar todas las filas. Considera las claves foráneas antes de eliminar.

Transacciones (opcional)


```
BEGIN TRANSACTION;  
UPDATE dbo.proyecto  
SET descripcion = N'Prueba temporal'  
WHERE id_proyecto = 1;  
SELECT id_proyecto, descripcion  
FROM dbo.proyecto  
WHERE id_proyecto = 1;  
ROLLBACK TRANSACTION;
```

ROLLBACK deshace los cambios de la transacción; COMMIT los confirma.

9. Ejercicios incrementales

Completa los campos. El indicador cuenta ejercicios con una respuesta escrita. Descarga el archivo antes de cerrar o recargar la página; las respuestas no se envían a ningún servidor.

Consultas y filtros básicos

01 Ejercicio 1. Exploración de proyectos

Consulta el identificador y el nombre de todos los proyectos.

Requisito: SELECT con columnas explícitas.

Consulta SQL / respuesta

```
Select id_proyecto, nombre_proyecto  
from proyecto;
```

Interpretación / observaciones

La consulta muestra el id y nombre de todos los proyectos

Consultas y filtros básicos

02 Ejercicio 2. Datasets externos

Consulta identificador, nombre y fuente de los datasets cuya fuente sea externa.

Requisito: WHERE y operador =.

Consulta SQL / respuesta

```
Select id_dataset, nombre, fuente  
from dataset  
Where fuente = 'externa';
```

Interpretación / observaciones

Select especifica cuales son las columnas que se van a ver en la consulta y where define que solo muestre aquellos registros con una fuente externa

Consultas y filtros básicos

03 Ejercicio 3. Métricas por umbral

Consulta métricas con valor mayor o igual a 0.80.

Requisito: Muestra identificador, nombre de métrica y valor; utiliza >=.

Consulta SQL / respuesta

```
Select id_metrica, nombre_metrica, valor
from metrica
where valor >= 0.80;
```

Interpretación / observaciones

Select escoge las columnas que quiero que muestre la consulta y el where especifica que solo los registros que tengan un valor mayor o igual a 0.80

Consultas y filtros básicos

04 Ejercicio 4. Rango de valores

Consulta métricas con valor entre 0.70 y 0.90, incluidos los extremos.

Requisito: Utiliza BETWEEN.

Consulta SQL / respuesta

```
Select id_metrica, nombre_metrica, valor
from metrica
where valor between 0.69 and 0.91;
```

Interpretación / observaciones

Select escoge las columnas que quiero que muestre la consulta y el where y between especifican que solo los valores que se encuentran en el rango. Sin, embargo, como no es posible incluir exactamente 0,70 y 0,90 y ponerlos directamente en el between se pone 0,69 y 0,91.

Consultas y filtros básicos

05 Ejercicio 5. Lista de fuentes

Consulta datasets cuya fuente pertenezca a interna o externa.

Requisito: Utiliza IN y explica el resultado.

Consulta SQL / respuesta

```
select *  
from dataset  
where fuente in ('interna' , 'externa');
```

Interpretación / observaciones

Se muestran todos los registros al solo existir fuente interna y externa

Patrones, NULL y lógica

06 Ejercicio 6. Búsqueda por prefijo

Consulta datasets cuyo nombre comienza por EEG.

Requisito: Utiliza LIKE y %.

Consulta SQL / respuesta

```
select *  
from dataset  
where nombre like ('EEG_');
```

Interpretación / observaciones

No se obtienen resultados debido a que ningun dataset empieza por EEG

Patrones, NULL y lógica

07 Ejercicio 7. Tamaños desconocidos

Consulta datasets con tamaño_filas desconocido y luego los que sí tienen tamaño.

Requisito: Utiliza IS NULL e IS NOT NULL.

Consulta SQL / respuesta

```
SELECT *  
FROM dataset  
where tamaño_filas IS NULL;
```

Interpretación / observaciones

Se obtienen resultados en ambas consultas debido a que son los únicos dos casos posibles

Patrones, NULL y lógica

08 Ejercicio 8. Condiciones combinadas

Consulta datasets externos cargados desde el 1 de enero de 2026.

Requisito: Utiliza AND y compara fechas válidas.

Consulta SQL / respuesta

```
SELECT *  
FROM dataset  
where fuente = 'externa' and fecha_carga >= '2026-01-01';
```

Interpretación / observaciones

Se obtienen resultados filtrados por medio del where. primero filtrando la fuente (al estar primero en la tabla) y luego la fecha exigiendo que se tenan en cuenta datasets cargados desde el 1 de enero de 2026

Patrones, NULL y lógica

09

Ejercicio 9. Negación

Consulta datasets cuya fuente no sea externa. Compara NOT con <>.

Requisito: Explica si las formas producen los mismos resultados.

Consulta SQL / respuesta

```
SELECT *  
FROM dataset  
where fuente = 'interna';
```

Interpretación / observaciones

La diferencia entre not y <> es que not suele utilizarse en campos null mientras que <> se utiliza para comparar dos valores

Ordenamiento y agregaciones

10

Ejercicio 10. Orden por fecha

Consulta nombre y fecha de carga, de más reciente a más antiguo.

Requisito: ORDER BY fecha_carga DESC.

Consulta SQL / respuesta

```
SELECT nombre, fecha_carga  
FROM dataset  
ORDER BY fecha_carga desc;
```

Interpretación / observaciones

El order by se encarga de ordenar todos los registros de la tabla según su fecha de carga de manera descendente es decir el mas nuevo primero y luego el mas viejo

Ordenamiento y agregaciones

11 Ejercicio 11. Orden por dos criterios

Consulta nombre, fuente y fecha; ordena fuente ascendente y fecha descendente.

Requisito: Dos criterios en ORDER BY.

Consulta SQL / respuesta

```
SELECT nombre, fuente, fecha_carga  
FROM dataset  
ORDER BY fuente asc, fecha_carga desc;
```

Interpretación / observaciones

EL order by permite ver primero todos los dataset de fuentes externas y posteriormente los ordena desde el mas reciente al mas viejo

Ordenamiento y agregaciones

12 Ejercicio 12. Conteo general

Calcula cuántos registros existen en dataset.

Requisito: COUNT(*) con alias claro.

Consulta SQL / respuesta

```
select count(*)  
from dataset;
```

Interpretación / observaciones

Esta consulta permite ver el total de registros que existen (100) sin tener que recorrer toda la tabla.

Ordenamiento y agregaciones

13

Ejercicio 13. Resumen de métricas

Calcula promedio, mínimo y máximo de `metrica.valor`.

Requisito: AVG, MIN y MAX con alias.

Consulta SQL / respuesta

```
select  avg(valor) as promedio
from metrica;

select  min(valor) as valor_minimo
from metrica;

select  max(valor) as valor_maximo
from metrica;
```

Interpretación / observaciones

Esta consulta permite entender correctamente cual es el promedio de los valores de `metrica` y cual es el valor maximo y minimo que se encuentra de entre todos los registros disponibles en la tabla

Ordenamiento y agregaciones

14

Ejercicio 14. Datasets por fuente

Calcula la cantidad de datasets por fuente.

Requisito: GROUP BY fuente y COUNT(*).

Consulta SQL / respuesta


```
select fuente, count(*)  
from dataset  
group by fuente;
```

Interpretación / observaciones

La consulta permite ver cuantos dataset de los registrados son internos y externos

Ordenamiento y agregaciones

15**Ejercicio 15. Promedio por métrica**

Calcula el promedio de valor para cada nombre_metrica.

Requisito: GROUP BY y AVG.

Consulta SQL / respuesta

```
select nombre_metrica, avg(valor) as promedio  
from metrica  
group by nombre_metrica;
```

Interpretación / observaciones

Se permite identificar cual es el promedio por cada tpo de metrica registrado en la tabla

Ordenamiento y agregaciones

16**Ejercicio 16. Filtrado de grupos**

Muestra tipos de métrica cuyo promedio supere 0.80.

Requisito: GROUP BY y HAVING; explica por qué AVG no va en WHERE.

Consulta SQL / respuesta

```
from metrica  
group by nombre_metrica  
Having avg(valor) > 0.80;
```

Interpretación / observaciones

Se permite filtrar cual tipo de metrica tiene un promedio de probabilidad de acierto mayor a 80%

Manipulación segura

17**Ejercicio 17. Crear un proyecto**

Inserta un proyecto con nombre único y descripción; después consúltalo.

Requisito: No indiques manualmente id_proyecto.

Consulta SQL / respuesta

```
insert into proyecto (nombre_proyecto, descripcion)
values ('Proyecto 1000', 'proyecto numero 1000 registrado en la base de
datos congratulations!!!!!!!');
```

Interpretación / observaciones

Se ingresan datos a la tabla sin incluir el id debido a que este es autoincremental

Manipulación segura

18**Ejercicio 18. Actualizar proyecto**

Modifica la descripción del proyecto del ejercicio 17.

Requisito: SELECT antes y después de UPDATE.

Consulta SQL / respuesta

```
where id_proyecto = 101,
update proyecto
set descripcion = 'proyecto perdedor no tuvo experimentos exitosos'
where id_proyecto = 101;
```

Interpretación / observaciones

Se verifico cual era la descripcion del proyecto del ejercicio 17 antes modificar su descripcion

Manipulación segura

19 Ejercicio 19. Eliminar proyecto de prueba

Elimina exclusivamente el proyecto creado en el ejercicio 17.

Requisito: SELECT antes y después de DELETE; no borres datos semilla.

Consulta SQL / respuesta

```
delete from proyecto  
where id_proyecto = 101;
```

Interpretación / observaciones

Antes del delete existia el id_proyecto 101 despues del delete no

Manipulación segura

20 Ejercicio 20. Calidad de métricas

Identifica tipos de métrica con promedio superior a 0.80 y presenta el promedio.

Requisito: Solo metrica, GROUP BY, AVG y HAVING; sin JOIN ni subconsultas.

Consulta SQL / respuesta

```
select nombre_metrica, avg(valor) as promedio  
from metrica  
group by nombre_metrica  
having avg(valor) > 0.80
```

Interpretación / observaciones

Se agrupan los datos por tipo de métrica, se calcula el promedio por grupos y se muestran

10. Preguntas de negocio

Responde con una consulta independiente por pregunta y una sola tabla.

1. ¿Cuántos datasets existen por fuente?

```
select fuente, count(*)  
from dataset  
group by fuente;
```

existen 50 datasets internos y 50 datasets externos

2. ¿Qué datasets tienen un tamaño de filas desconocido?

```
SELECT *  
FROM dataset  
where tamaño_filas IS NULL;
```

El resultado son 10 registros

3. ¿Cuál es el tamaño máximo informado entre los datasets?

```
SELECT MAX(tamaño_filas)  
from dataset;
```

El mayor tamaño de filas informado es 123750

4. ¿Cuál es el promedio de cada tipo de métrica?

```
select nombre_metrica, avg(valor) as promedio
from metrica
group by nombre_metrica;
```

accuracy tiene un promedio de: 0.7927
y precision tiene un promedio de: 0.7150

5. ¿Qué tipos de métrica superan un promedio de 0.80?

```
select nombre_metrica, avg(valor) as promedio
from metrica
group by nombre_metrica
having avg(valor) > 0.80;
```

Ninguna metrica tiene un valor mayor a 0.80

6. ¿Cuántos modelos existen por algoritmo?

```
select algoritmo, count(*)
from modelo
group by algoritmo;
```

existen 20 registros por tipo de algoritmo

7. ¿Qué modelos tienen una versión que comienza por 1.2?

```
select *
from modelo
where version like ('1.2%');
```

hay 20 registros que su version inicia con 1.2

Checklist de entrega

- La semilla se ejecutó y cada tabla tiene al menos 100 filas.
- Los ejercicios usan consultas de una sola tabla.
- No se usaron JOIN, subconsultas ni operadores de combinación.
- UPDATE y DELETE tienen verificación previa y posterior.
- Las consultas se guardaron en scripts/consultas/s08-filtros-orden-agregacion.sql.
- Los scripts están versionados en Git.

La guía es autónoma y no envía respuestas a un servidor. Usa la descarga para conservar el trabajo.